Redes neuronales básicas

1. Estructura neuronal básica con Keras: ¿Cómo se define y configura una arquitectura secuencial utilizando tf.keras.Sequential y capas Dense, y qué parámetros clave deben establecerse al especificar la dimensión de entrada (input_shape) y el número de neuronas por capa?

Se crea con **tf.keras.Sequential** y se agregan capas **Dense** una después de otra. **input_shape** indica cuántos datos recibe la red; por ejemplo, (4,) significa cuatro valores de entrada. En cada capa Dense se elige el número de neuronas y su función de activación.

2. Funciones de activación en capas ocultas: ¿De qué manera influye la elección de una función de activación (como ReLU, Leaky ReLU o Tanh) en las capas internas sobre la capacidad del modelo para aprender relaciones no lineales y evitar el desvanecimiento del gradiente?

Estas funciones permiten aprender patrones que no siguen una línea recta. **ReLU** es sencilla y suele facilitar el aprendizaje; **Leaky ReLU** deja pasar una pequeña señal cuando el valor es negativo, y **Tanh** produce valores entre -1 y 1. Tanh puede hacer que el aprendizaje sea más lento si sus gradientes se vuelven muy pequeños.

3. Predicción (Regresión numérica): Al diseñar una red neuronal orientada a la estimación de valores numéricos continuos, ¿qué configuración particular debe llevar la capa de salida (número de neuronas y activación) y qué función de pérdida es la adecuada para medir el margen de error?

Para predecir un solo número, como un precio o una temperatura, la salida normalmente tiene **una neurona con activación lineal** (o sin indicar activación). Para medir qué tan lejos está la predicción del valor real, se suele utilizar **mean_squared_error (MSE)**; también se puede usar MAE.

4. Clasificación binaria: ¿Por qué en un problema de clasificación binaria es estándar utilizar la función de activación Sigmoid en la neurona de salida junto con la función de pérdida binary_crossentropy, y cómo se interpreta la probabilidad emitida por el modelo?

Sigmoid devuelve un valor entre 0 y 1 que representa la probabilidad de una de las dos clases. **binary_crossentropy** mide qué tan equivocada está esa probabilidad durante el entrenamiento. Por ejemplo, una salida de 0.8 indica un 80 % de probabilidad para la clase 1; normalmente se usa 0.5 como límite para decidir.

5. Clasificación multiclase: ¿Cómo transforma la función de activación Softmax las salidas (logits) de la red en una distribución de probabilidad para múltiples clases, y qué criterio determina si se debe compilar el modelo con categorical_crossentropy o sparse_categorical_crossentropy?

Softmax convierte los resultados de la red en probabilidades para todas las clases; juntas suman 100 %. Se usa **categorical_crossentropy** si las etiquetas están en formato one-hot, por ejemplo [0, 1, 0], y **sparse_categorical_crossentropy** si son números enteros, como 0, 1 o 2.

6. Normalización de datos de entrada: ¿Por qué resulta indispensable escalar o normalizar las características de entrada antes de alimentar una red neuronal en TensorFlow, y qué impacto directo tiene este procesamiento en la estabilidad y rapidez de la convergencia del entrenamiento?

Normalizar consiste en poner los datos en escalas parecidas, por ejemplo entre 0 y 1. Así se evita que los valores muy grandes dificulten el aprendizaje y, por lo general, la red entrena de manera más estable y llega a buenos resultados más rápido.

7. Codificación one-hot (one-hot encoding): ¿En qué consiste la técnica de codificación one-hot para representar etiquetas categóricas, qué métodos ofrece TensorFlow/Keras para aplicarla y en qué casos es preferible frente a un esquema de enteros?

Convierte cada categoría en una lista de ceros con un solo uno. Por ejemplo, para tres clases, la segunda sería [0, 1, 0]. Puede hacerse con **tf.one_hot()** o **tf.keras.utils.to_categorical()**. Es útil si se utiliza categorical_crossentropy; si las etiquetas son enteros, sparse_categorical_crossentropy evita convertirlas.

8. Funciones de pérdida (loss functions): ¿De qué manera actúa la función de pérdida como guía matemática para ajustar los parámetros del modelo durante el aprendizaje y cuáles son las consecuencias de seleccionar una función no alineada con el tipo de tarea?

La función de pérdida calcula cuánto se equivoca la red. Con ese error, el entrenamiento ajusta los pesos para intentar mejorar las siguientes predicciones. Si se elige una función que no corresponde al problema, el modelo puede aprender mal o dar resultados poco útiles.

9. Fase de entrenamiento (model.fit): ¿Investigar qué secuencia de pasos matemáticos e internos (propagación hacia adelante, cálculo del error y retropropagación o backpropagation) ocurren durante cada época (epoch) de entrenamiento dentro del método model.fit()?

Primero, la red recibe los datos y produce una predicción (**propagación hacia adelante**). Luego calcula el error con la función de pérdida. Después, la **retropropagación** calcula cómo debe cambiar cada peso, y el optimizador lo actualiza. Esto se repite por grupos de datos; una época termina cuando se ha recorrido todo el conjunto de entrenamiento.

10. Entorno de ejecución en Google Colab: ¿Qué ventajas técnicas ofrece Google Colab para el desarrollo de proyectos con TensorFlow (como el soporte de GPU/TPU) y qué procedimientos se requieren para gestionar la persistencia de datos y el almacenamiento de modelos entrenados?

Google Colab permite programar en Python desde el navegador sin configurar todo en la computadora. Puede ofrecer GPU o TPU para acelerar ciertos entrenamientos, según la disponibilidad. Como sus sesiones son temporales, conviene conectar Google Drive para guardar los datos y guardar el modelo con **model.save()**, por ejemplo en formato .keras.

11. Evaluación y diagnóstico del rendimiento: Tras finalizar el entrenamiento, ¿de qué forma se evalúa la capacidad de generalización del modelo mediante model.evaluate() y model.predict(), y cómo se analizan las curvas de pérdida y exactitud (accuracy) para detectar sobreajuste (overfitting) o subajuste (underfitting)?

model.evaluate() mide el error y las métricas con datos que no se usaron para entrenar, mientras que **model.predict()** genera predicciones. Si el entrenamiento mejora pero la validación empeora, puede haber **sobreajuste**. Si ambos resultados siguen siendo malos, puede haber **subajuste**. Las gráficas de pérdida y accuracy ayudan a notarlo.